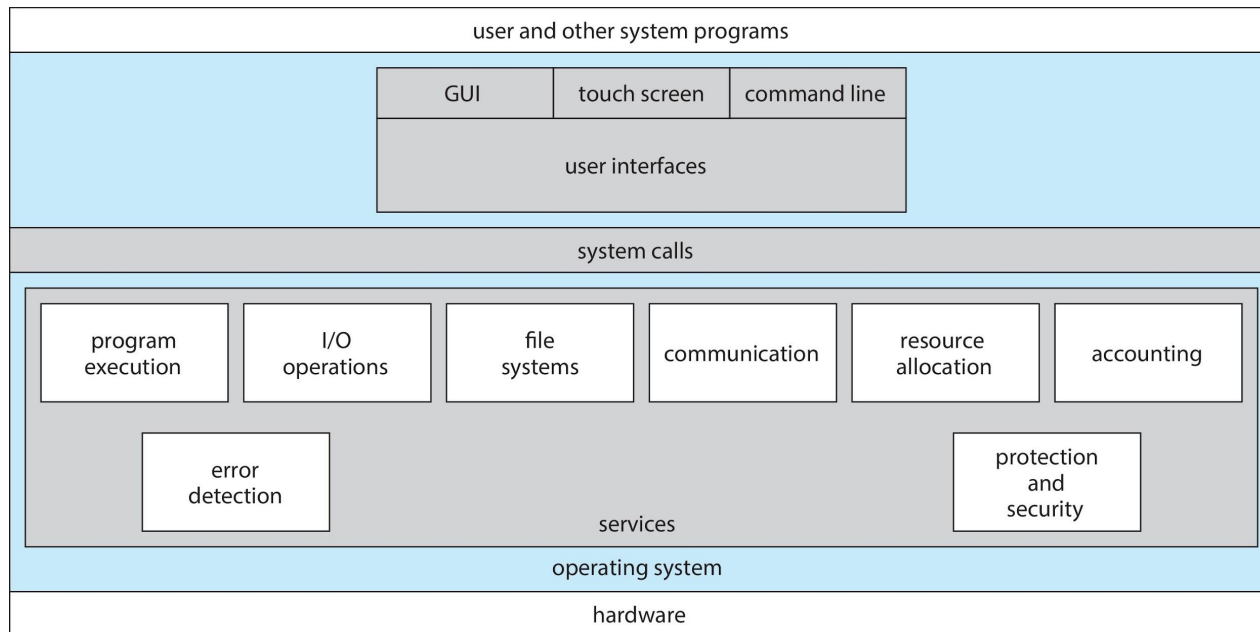
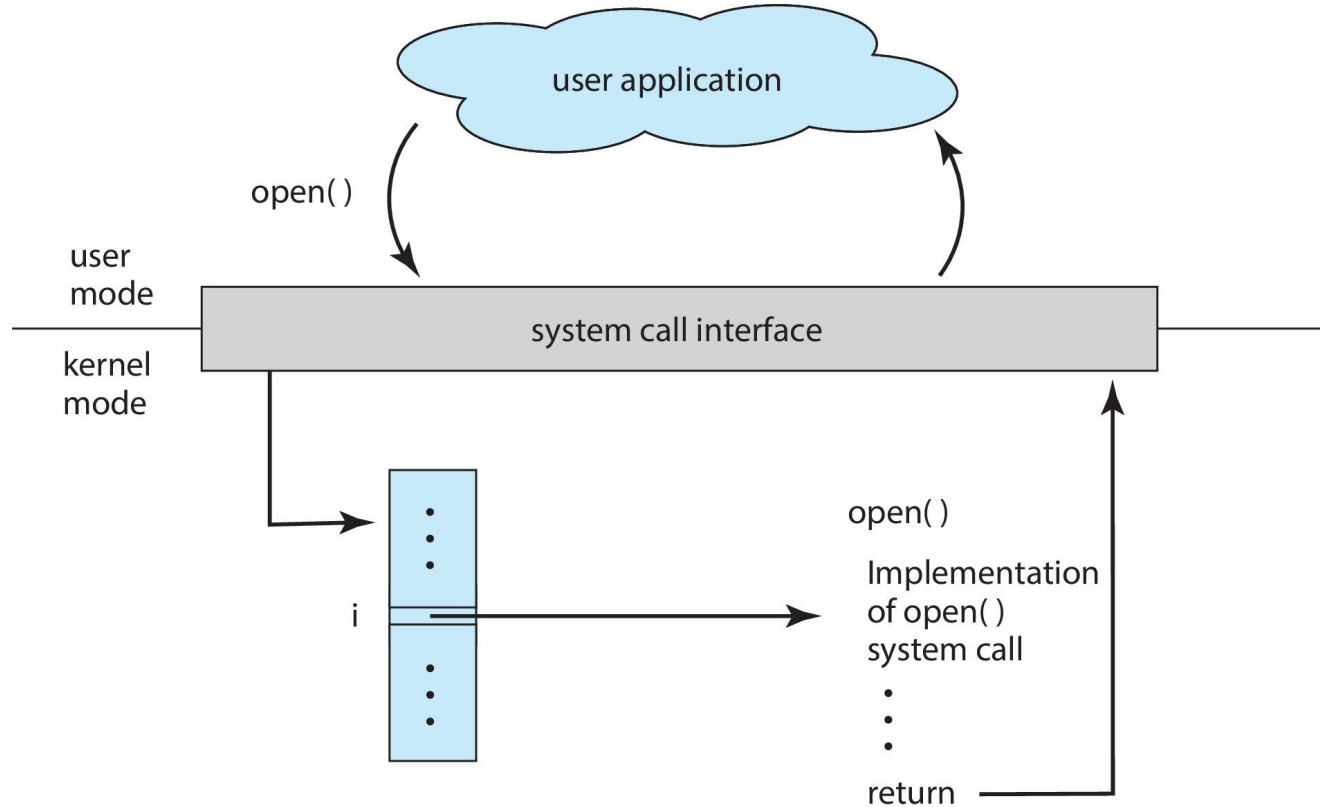


Review: A View of Operating System Services



Review: API – System Call – OS Relationship



Review: System Call Parameter Passing

1. **pass the parameters in registers**

- Simplest (no context copy)
- In some cases, may be more parameters than registers

2. **address of a block passed as a parameter in a register**

- Parameters stored in the block, or table, in memory
- This approach taken by Linux and Solaris

3. **Parameters placed, or pushed, onto the stack by the program**

- **and popped off the stack by OS**

Block and stack methods do not limit the number or length of parameters being passed

- however they use memory

Review: kernel implementation structure

	monolithic kernel	microkernel
Reliability	If one driver crashes, entire kernel fails	Kernel can restart system program as needed
Ease of Development	Must get entire kernel to work	Able to test just one part without affecting rest
Speed	faster: No extraneous context switching,	Slow: due to message passing (context switches)
memory	Relatively modest in memory usage (run in single address space)	Memory footprint much larger (separate address spaces)

hybrid kernel

- In practice, modern OSes are hybrid
- Linux is more monolithic than current Windows and macOS
 - [Linux kernel diagram](#)
 - [Architecture of macOS - Wikipedia](#)
 - [Overview of Windows Components - Windows drivers | Microsoft Learn](#)
- Loadable kernel modules: bit of code that kernel can load (and usually unload) while system is running, to extend functionality
- Examples:
 - Linux kernel modules (lkm),
 - Windows device drivers
 - macOS extension

Process Management

Chapter 3: Processes

- Process Concept
- Process Scheduling
- Operations on Processes
- Interprocess Communication
- IPC in Shared-Memory Systems
- IPC in Message-Passing Systems
- Examples of IPC Systems
- Communication in Client-Server Systems

User view of a process

Process in Memory

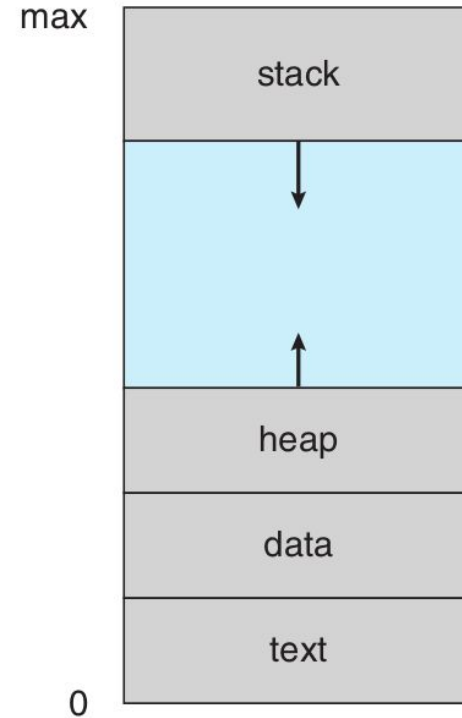
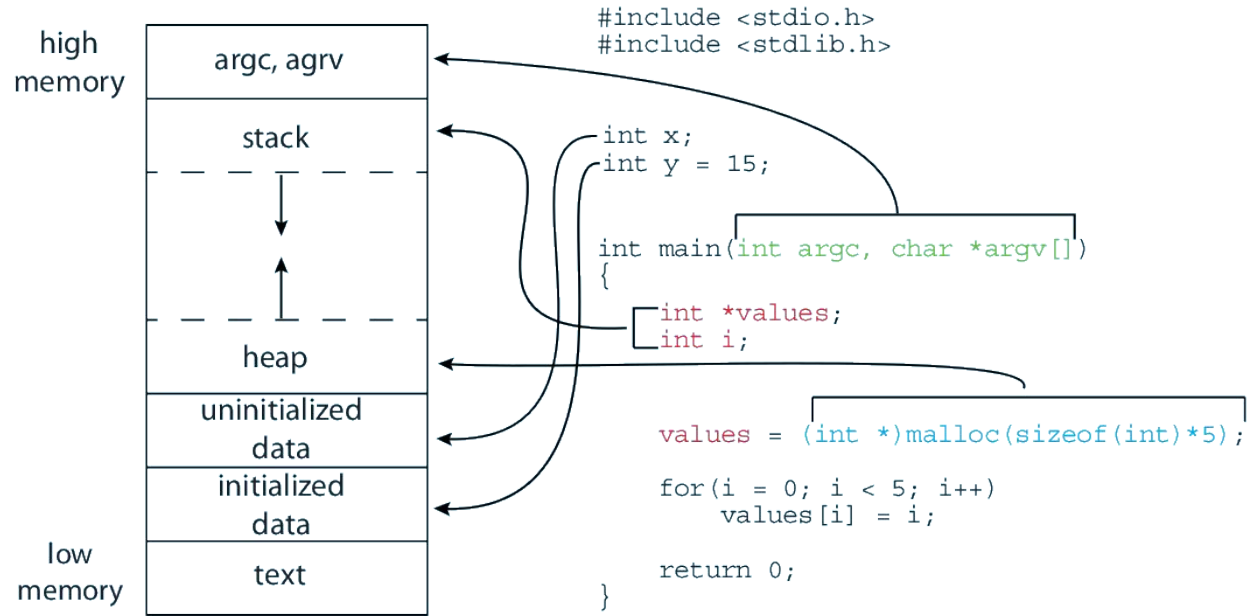


Figure 3.1 Layout of a process in memory.

Memory Layout of a C Program



```
$ gcc memory.c -o memory
```

```
$ size memory
```

text	data	bss	dec	hex	filename
1603	600	8	2211	8a3	memory

note: stack and heap runtime segments

Explanations for the previous diagrams

- An operating system executes a variety of programs that run as a process.
- **Process** – a program in execution; process execution must progress in sequential fashion. No parallel execution of instructions of a single process
- Multiple parts
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data
 - Function parameters, return addresses, local variables
 - **Data section** containing global variables
 - **Heap** containing memory dynamically allocated during run time
- Program is **passive** entity stored on disk (**executable file**); process is **active**
 - Program becomes process when an executable file is loaded into memory
- Execution of program started via GUI mouse clicks, command line entry of its name, etc.
- One program can be several processes
 - Consider multiple users executing the same program

```

#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>

int global_j; const int ci = 24;

void main (int argc, char **argv){
    int local_stack = 0;
    char *const_data = "This data is constant";
    char *tiny = malloc (32); /* allocate 32 bytes */
    char *small = malloc (2*1024); /* Allocate 2K */
    char *large = malloc (1*1024*1024); /* Allocate 1MB */

    printf ("Text is %p\n", main);
    printf ("Global Data is %p\n", &global_j);
    printf ("Local (Stack) is %p\n", &local_stack);
    printf ("Constant data is %p\n",&ci );
    printf ("Hardcoded string (also constant) are at %p\n", const_data );
    printf ("Tiny allocations from %p\n", tiny);
    printf ("Small allocations from %p\n", small);
    printf ("Large allocations from %p\n", large);
    printf ("Malloc (i.e. libSystem) is at %p\n", malloc);
    sleep(100); /* so we can use vmmap on this process before it exits */
}

```

\$./a.out &

[2] 9584

Text is 0x55b81357e149

Global Data is 0x55b813581024

Local (Stack) is 0x7ffdfdb6e24c

Constant data is 0x55b81357f008

Hardcoded string (also constant) are at 0x55b81357f00c

Tiny allocations from 0x55b8147992a0

Small allocations from 0x55b8147992d0

Large allocations from 0x7fca29d15010

Malloc (i.e. libSystem) is at 0x7fca29eb1870

\$./a.out &

[2] 9584

Text is 0x55b81357e149

Global Data is 0x55b813581024

Local (Stack) is 0x7ffdfdb6e24c

Constant data is 0x55b81357f008

Hardcoded string (also constant) are at 0x55b81357f00c

Tiny allocations from 0x55b8147992a0

Small allocations from 0x55b8147992d0

Large allocations from 0x7fca29d15010

Malloc (i.e. libSystem) is at 0x7fca29eb1870

\$ pmap -x 9584 #vmmap in macos

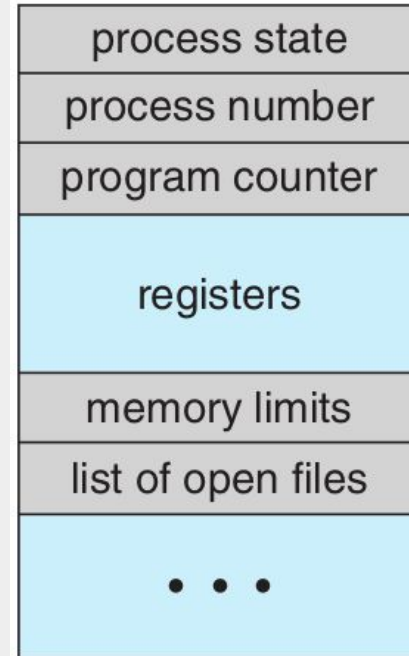
9584: ./a.out

Address	Kbytes	RSS	Dirty	Mode	Mapping
000055b81357d000	4	4	0	r----	a.out
000055b81357e000	4	4	0	r-x--	a.out
000055b81357f000	4	4	0	r----	a.out
000055b813580000	4	4	4	r----	a.out
000055b813581000	4	4	4	rw---	a.out
000055b814799000	132	4	4	rw---	[anon]
00007fca29d15000	1040	12	12	rw---	[anon]
00007fca29e19000	152	148	0	r----	libc.so.6
00007fca29e3f000	1364	892	0	r-x--	libc.so.6
00007fca29f94000	332	128	0	r----	libc.so.6
00007fca29fe7000	16	16	16	r----	libc.so.6
00007fca29feb000	8	8	8	rw---	libc.so.6
00007fca29fed000	52	20	20	rw---	[anon]
00007fca2a00e000	8	4	4	rw---	[anon]
00007fca2a010000	4	4	0	r----	ld-linux-x86-64.so.2
00007fca2a011000	148	148	0	r-x--	ld-linux-x86-64.so.2
00007fca2a036000	40	36	0	r----	ld-linux-x86-64.so.2
00007fca2a040000	8	8	8	r----	ld-linux-x86-64.so.2
00007fca2a042000	8	8	8	rw---	ld-linux-x86-64.so.2
00007ffdfdb4f000	132	16	16	rw---	[stack]
00007ffdfdbf5000	16	0	0	r----	[anon]
00007ffdfdbf9000	8	4	0	r-x--	[anon]

total kB 3488 1476 104

Kernel view of process

Process Control Block (PCB)



Process States

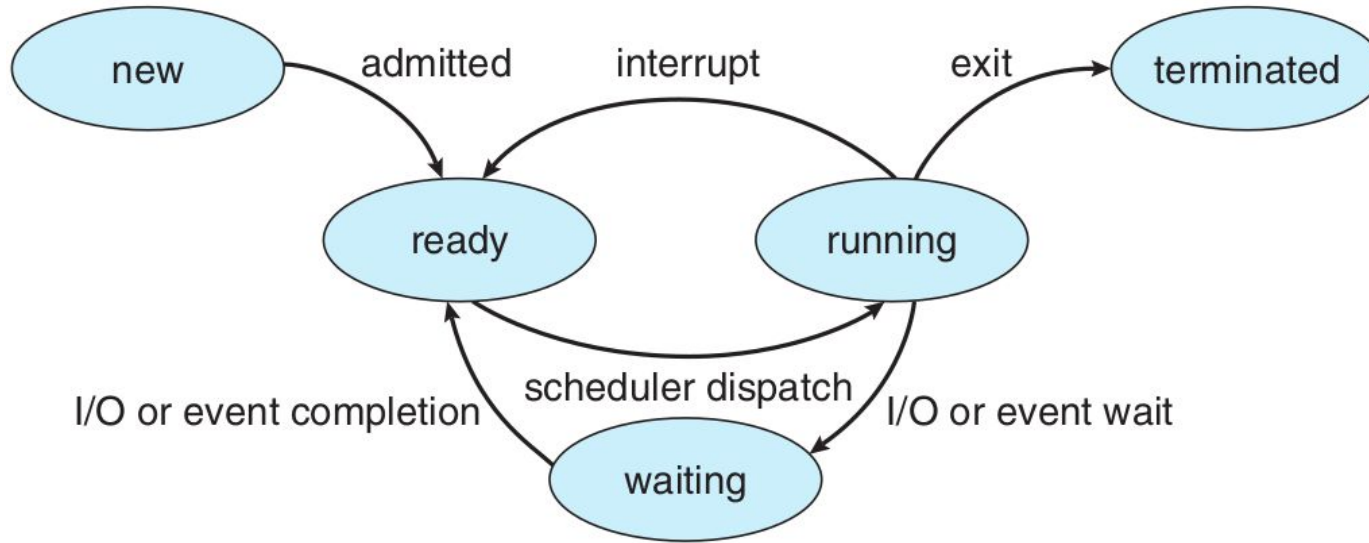


Figure 3.2 Diagram of process state.

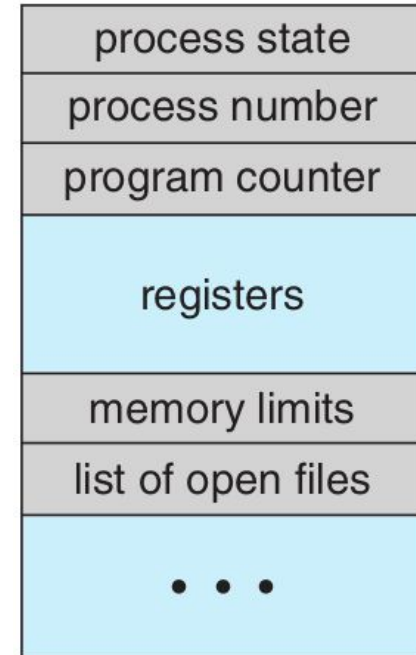
As a process executes, it changes **state**

- **New:** The process is being created
- **Running:** Instructions are being executed
- **Waiting:** The process is waiting for some event to occur
- **Ready:** The process is waiting to be assigned to a processor
- **Terminated:** The process has finished execution

Process Control Block (PCB)

Information associated with each process(also called **task control block**)

- Process state
 - – running, waiting, etc.
- Program counter
 - – location of instruction to next execute
- CPU registers
 - – contents of all process-centric registers
- CPU scheduling information
 - - priorities, scheduling queue pointers
- Memory
 - -management information – memory allocated to the process
- Accounting information
 - – CPU used, clock time elapsed since start, time limits
- I/O status information
 - – I/O devices allocated to process, list of open files



The concept of threads

- The process model so far performs **a single thread of execution**.
- Consider having multiple locations to execute at once
 - Multiple threads of control -> **threads**

The concept of threads

- Consider having **multiple program counters per process**
- Multiple locations can execute at once
 - Multiple threads of control
-> **threads**
- Must then have storage for thread details
 - **multiple program counters in PCB**

Process Representation in Linux

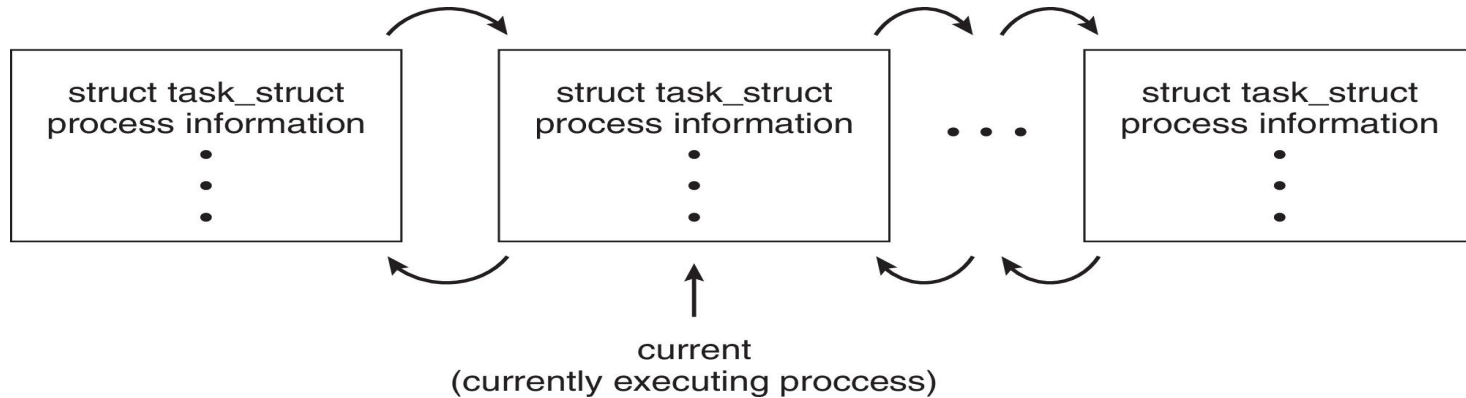
Represented by the C structure

```
struct task_struct {  
    struct thread_info    thread_info;  
    unsigned int          __state;  
    unsigned int          saved_state;  
    void                  *stack;  
    refcount_t            usage;  
    unsigned int          flags;  
    unsigned int          ptrace;  
    struct alloc_tag      *alloc_tag;  
    int                   on_cpu;  
    struct task_struct    *last_wakee;  
    int                   recent_used_cpu;  
    int                   wake_cpu;  
    int                   on_rq;  
    int                   prio;  
    int                   static_prio;  
    int                   normal_prio;  
    unsigned int          rt_priority;  
    struct sched_entity   se;  
    struct sched_rt_entity rt;  
};
```

```
    struct list_head      tasks;  
    struct mm_struct      *mm;  
  
    /* Real parent process: */  
    struct task_struct    __rcu *real_parent;  
  
    /* Recipient of SIGCHLD, wait4() reports: */  
    struct task_struct    __rcu *parent;  
  
    /*  
     * Children/sibling form the list of natural  
     * children:  
     */  
    struct list_head      children;  
    struct list_head      sibling;
```

[include/linux/sched.h - Linux source code v6.13 - Bootlin Elixir Cross Referencer](#)

```
#define for\_each\_process(p) \
    for (p = &init\_task ; (p = next\_task(p)) != &init\_task ; )
```



/ To Iterate */*

```
struct task_struct *task;
for_each_process(task) {
    int state = READ_ONCE((task)->__state)
```

```
task\_is\_running(task)
task\_is\_traced(task)
task\_is\_stopped(task)
task\_is\_stopped\_or\_traced(task)
is\_special\_task\_state(state)
```

```
WRITE\_ONCE(current->__state, (state_value));
```

Process Scheduling

Process scheduler selects among available processes for next execution on CPU core

Goal?

Process Scheduling

Process scheduler

selects among
available processes for
next execution on CPU
core

- Goal
 - Maximize CPU use (**CPU utilization**),
 - quickly switch processes onto CPU core
 - Switch a CPU core among processes so frequently that users can interact with each program while it is running .

Process Scheduling-implementation

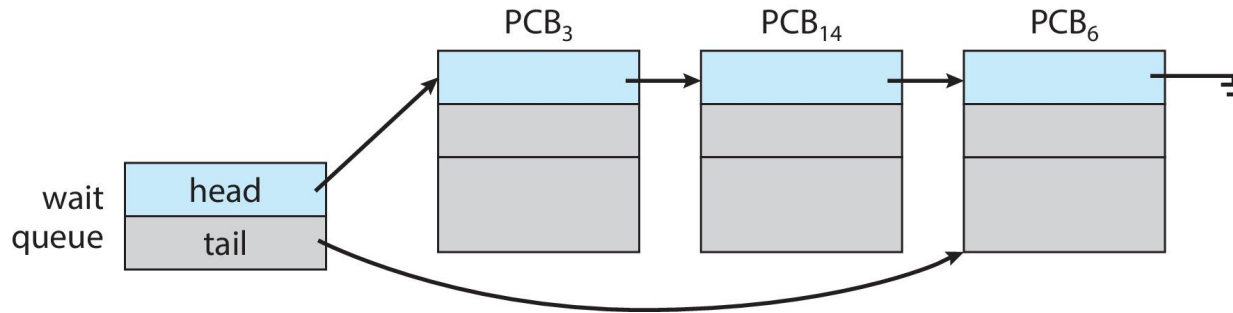
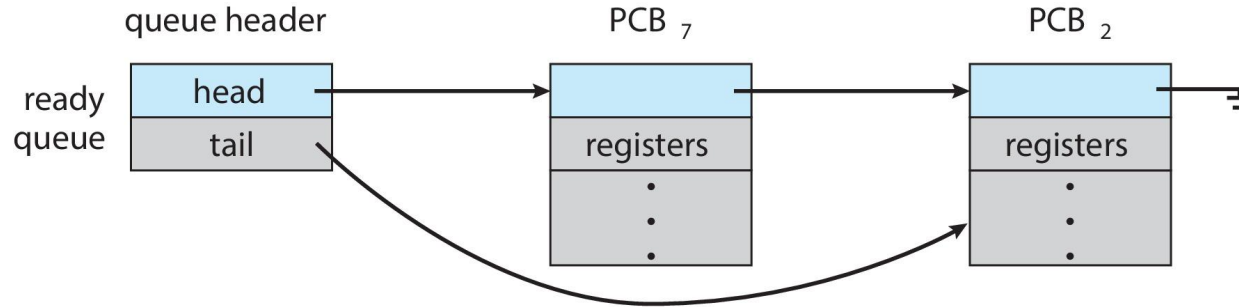
Maintains **scheduling queues** of processes

- **Ready queue** – set of all processes residing in main memory, ready and waiting to execute
- **Wait queues** – set of processes waiting for an event (i.e., I/O)

Processes migrate among the various queues

ready queue

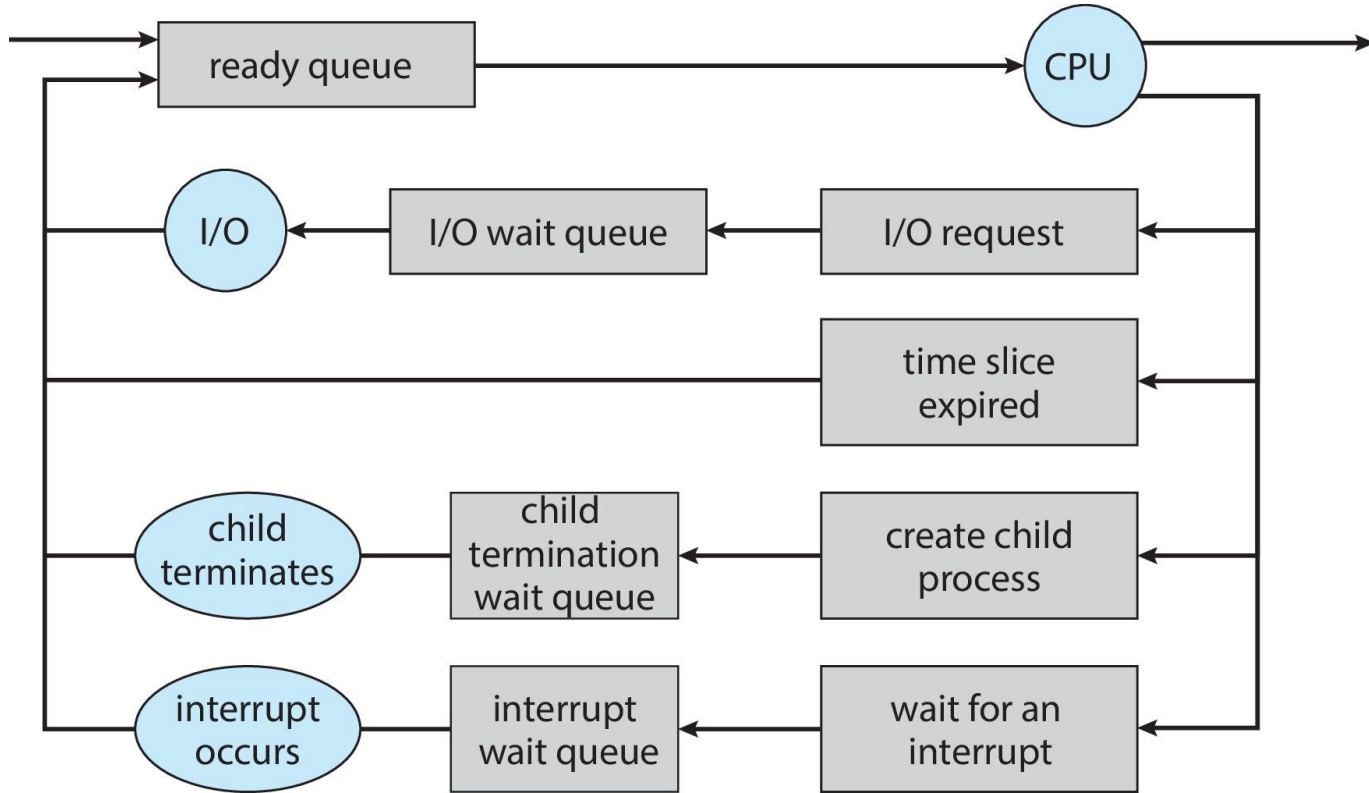
- As processes enter the system, they are put into this queue,
- Processes in this queue are ready and waiting to execute on a CPU's core



wait queue

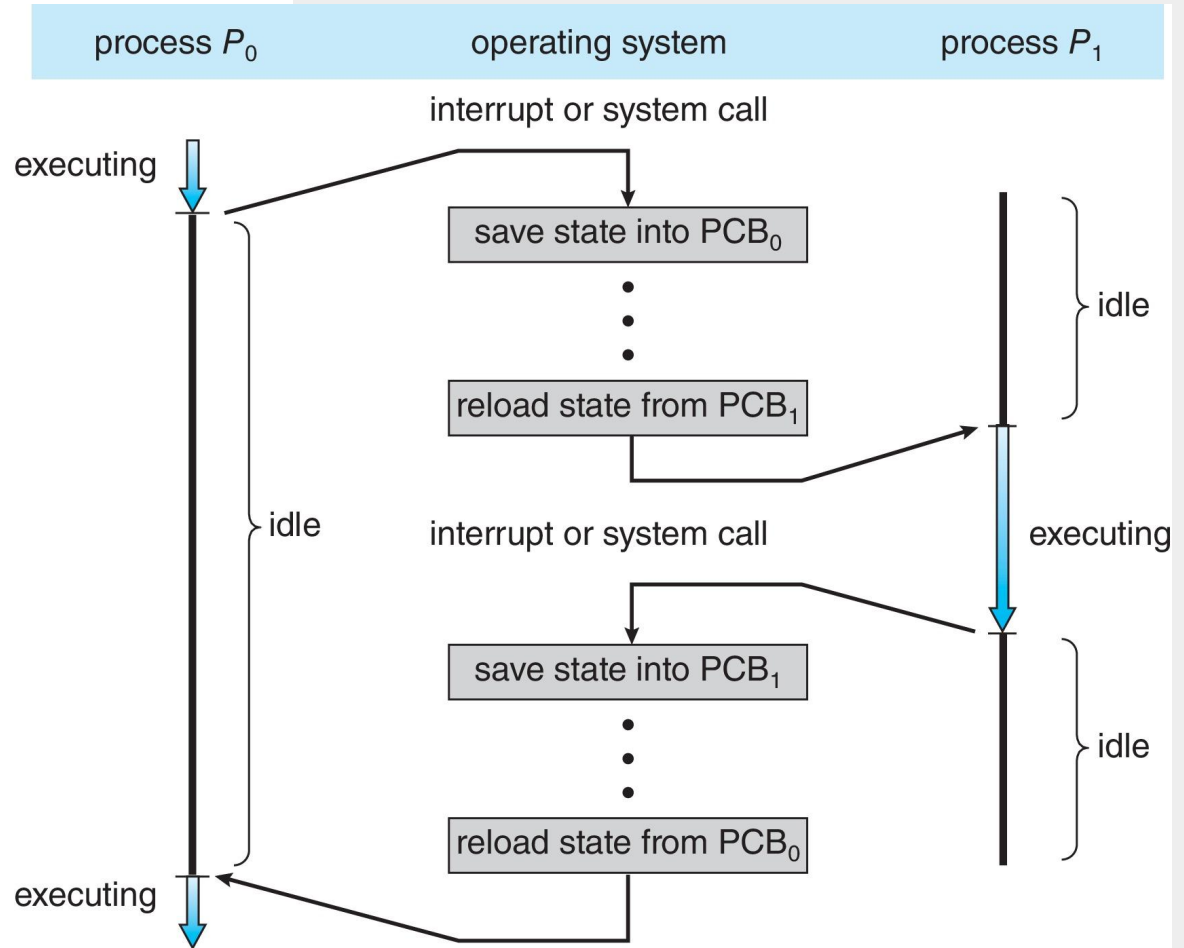
- Processes that are waiting for a certain event to occur — such as completion of I/O — are placed in a

Process Scheduling



Switching CPU From One Process to Another

A **context switch** occurs when the CPU switches from one process to another.



Multitasking in Mobile Systems

- Some mobile systems (e.g., early version of iOS) allow only one process to run, others suspended
-
- iOS provides for a
 - Single **foreground** process- controlled via user interface
 - Multiple **background** processes– in memory, running, but not on the display, and with limits
 - Limits include single, short task, receiving notification of events, specific long-running tasks like audio playback

A book on macOS

<https://newosxbook.com/home.html>

<https://newosxbook.com/jbooks.html>

<https://newosxbook.com/MOXil.pdf>

Android details

- Android runs foreground and background, with fewer limits ([Processes and app lifecycle | Android Developers](#))
- When deciding how to classify a process, the system bases its decision on the most important level found among all the components currently active in the process (see [Activity](#), [Service](#), and [BroadcastReceiver](#)).
- [Activity](#), [Service](#), and [BroadcastReceiver](#) impact the lifetime of the application's process.
 - system determines which processes to kill when low on memory

Based on [Activity](#), [Service](#), and [BroadcastReceiver](#)

- **Foreground, visible, service, or cached process**

android details:

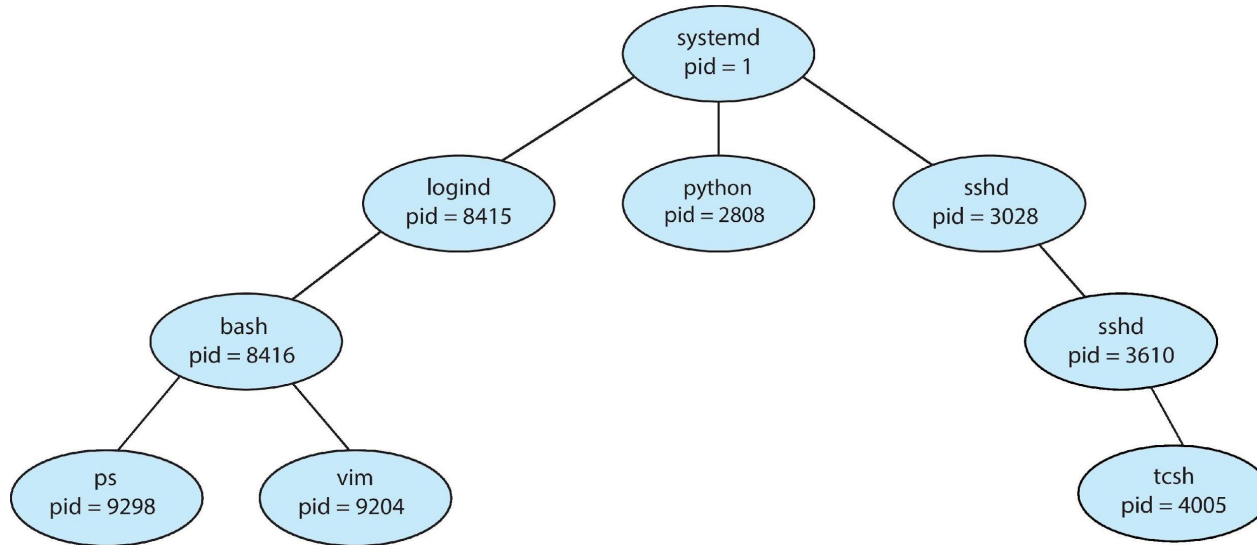
[Processes and app lifecycle | Android Developers](#)

Operations on Processes

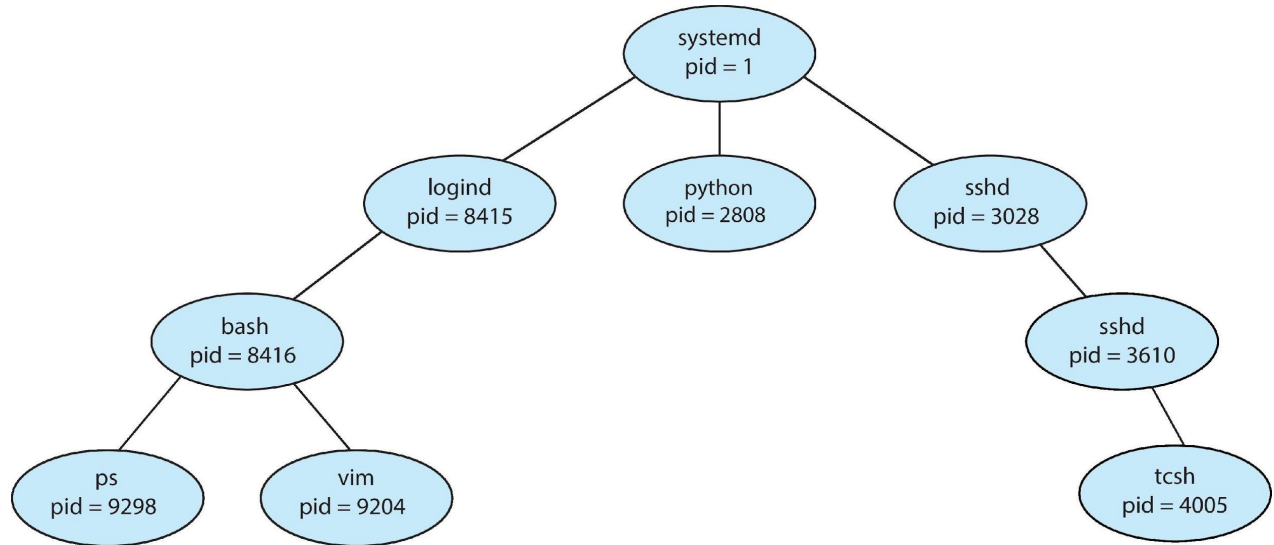
- System must provide mechanisms for:
 - Process creation
 - Process termination

Process Creation

- Generally, process identified and managed via a **process identifier (pid)**
- **Parent** process create **children** processes, which, in turn create other processes, forming a **tree** of processes



- **Resource sharing options**
 - Parent and children share all resources
 - Children share subset of parent's resources
 - Parent and child share no resources
- **Execution options**
 - Parent and children execute concurrently
 - Parent waits until children terminate



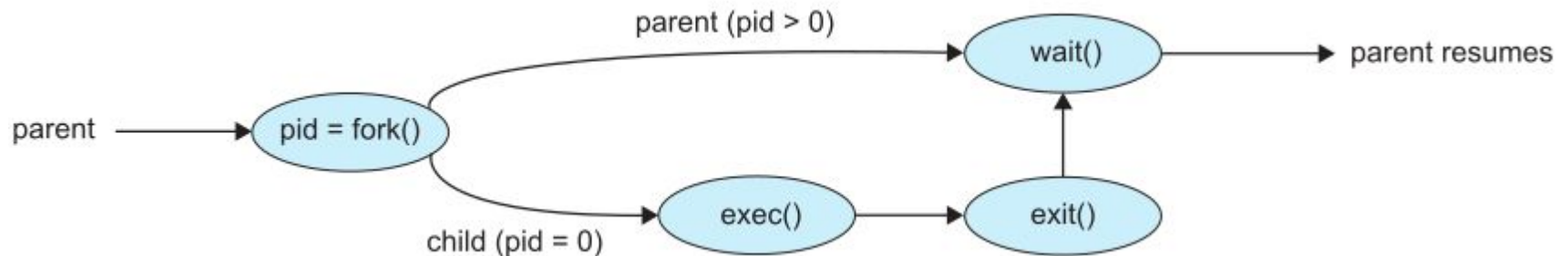
Process Creation (Cont.)

- Address space

- Child duplicate of parent
- Child has a program loaded into it

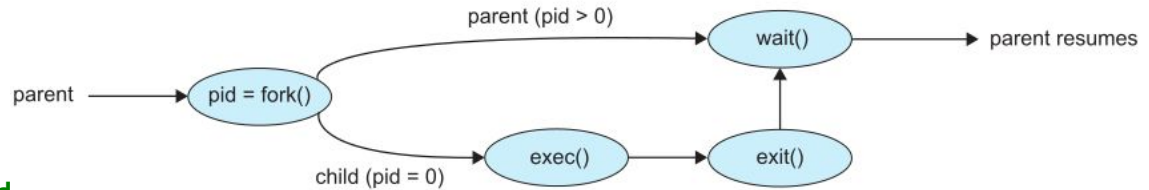
- UNIX examples

- **fork()** system call creates new process
- **exec()** system call used after a **fork()** to replace the process' memory space with a new program
- Parent process calls **wait()** waiting for the child to terminate



Unix example: C Program Forking Separate Process

```
#include < sys/types.h >
#include < stdio.h >
#include < unistd.h >
int main(){
    pid_t pid;
    /* fork a child process */
    pid = fork();
    if (pid < 0){ /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0){ /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else{ /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete");
    }
    return 0;
}
```



see also `$ man -S2 clone3`

Creating a Separate Process via Windows API

```
#include < stdio.h >
#include < windows.h >
int main(VOID) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    /* allocate memory */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    ZeroMemory(&pi, sizeof(pi));
    /* create child process */
    if (!CreateProcess(NULL, /* use command line */
        "C: \\ WINDOWS \\ system32 \\ mspaint.exe", /* command */
        NULL, /* don't inherit process handle */
        NULL, /* don't inherit thread handle */
        FALSE, /* disable handle inheritance */
        0, /* no creation flags */
        NULL, /* use parent's environment block */
        NULL, /* use parent's existing directory */
        &si, &pi)) {
        fprintf(stderr, "Create Process Failed");
        return -1;
    }
    /* parent will wait for the child to complete */
    WaitForSingleObject(pi.hProcess, INFINITE);
    printf("Child Complete");

    /* close handles */
    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);
}
```

Process Termination

- Process executes last statement and then asks the operating system to delete it using the **exit()** system call.
 - Returns status data from child to parent (via **wait()**)
 - Process' resources are deallocated by operating system

Process Termination

- Parent may terminate the execution of children processes using the `kill()` system call.
- Some reasons for doing so:
 - Child has exceeded allocated resources
 - Task assigned to child is no longer required
 - The parent is exiting, and the operating systems does not allow a child to continue if its parent terminates

Process Termination

- Some operating systems do not allow child to exist if its parent has terminated.
 - If a process terminates, then all its children must also be terminated.
 - **cascading termination.** All children, grandchildren, etc., are terminated.
 - The termination is initiated by the operating system.

Process Termination

- The parent process may wait for termination of a child process by using the `wait()` system call.

```
pid = wait(&status);
```

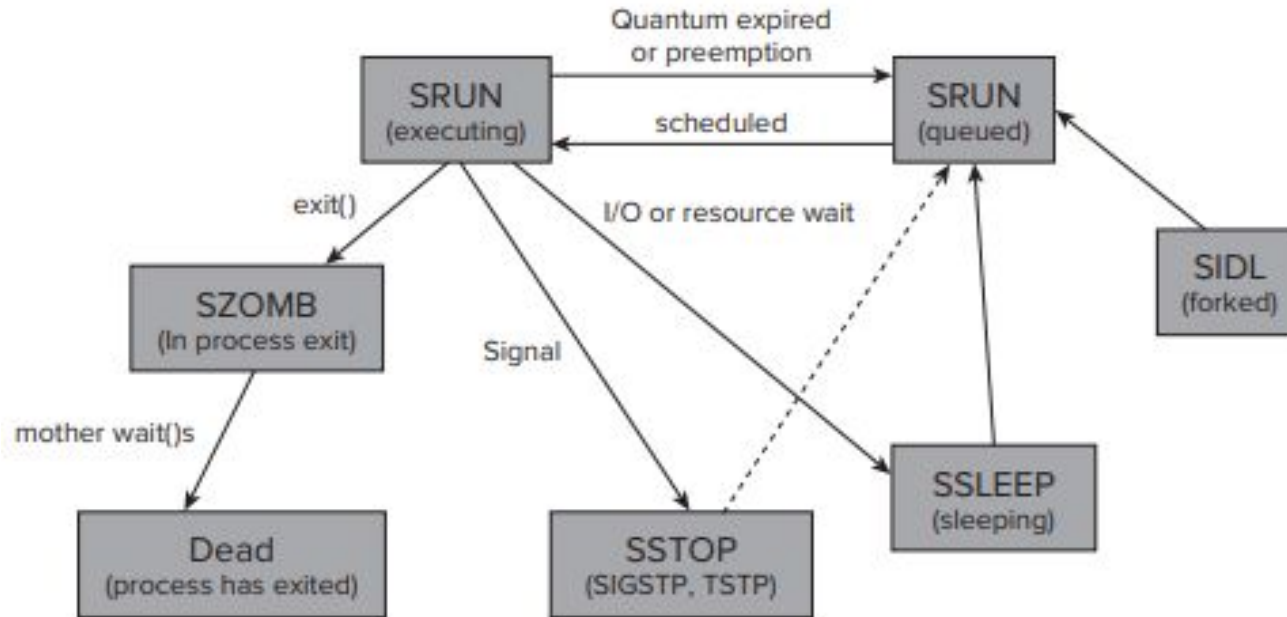
- The call returns status information and the pid of the terminated process

zombie process: If no parent waiting (did not invoke `wait()`) process is a **zombie process**

- **PCB remains**

orphan process: If parent terminated without invoking `wait()`, child-process is **orphan**

XNU process life cycle



<https://github.com/apple-oss-distributions/xnu/blob/main/bsd/sys/proc.h>

fig4.1 in <https://newosxbook.com/MOXil.pdf>

Zombie example

```
#include <unistd.h>
#include <stdio.h>
int main(int argc, char **argv){
    int rc = fork(); /* This returns twice*/
    int child = 0;
    switch (rc) {
        case -1:
            fprintf(stderr, "Unable to fork!\n");
            return (1);
        case 0:
            printf("I am the child! I am born id:%d\n", getpid());
            child++;
            break;
        default:
            printf("I am the parent! Going to sleep and now wait()ing\n");
            sleep(60);
    }
    printf("%s exiting\n", (child ? "child" : "parent"));
    return (0);
}
```

```
$ ./a.out &
$ ps a
```

Process Termination based on Importance Hierarchy

- Mobile operating systems often have to terminate processes to reclaim system resources such as memory.
- Android will begin terminating processes that are least important.
 - **the states are saved before the termination**
- From **most** to **least** important:
 - Foreground process
 - Visible process
 - Service process
 - Background process
 - Empty process

Interprocess Communication (IPC)

- Processes within a system may be *independent* or *cooperating*
- Cooperating process can affect or be affected by other processes, including sharing data
- Reasons for cooperating processes:
 - Information sharing
 - Computation speedup
 - Modularity
 - Convenience
- Cooperating processes need **interprocess communication (IPC)**

Why need IPC?

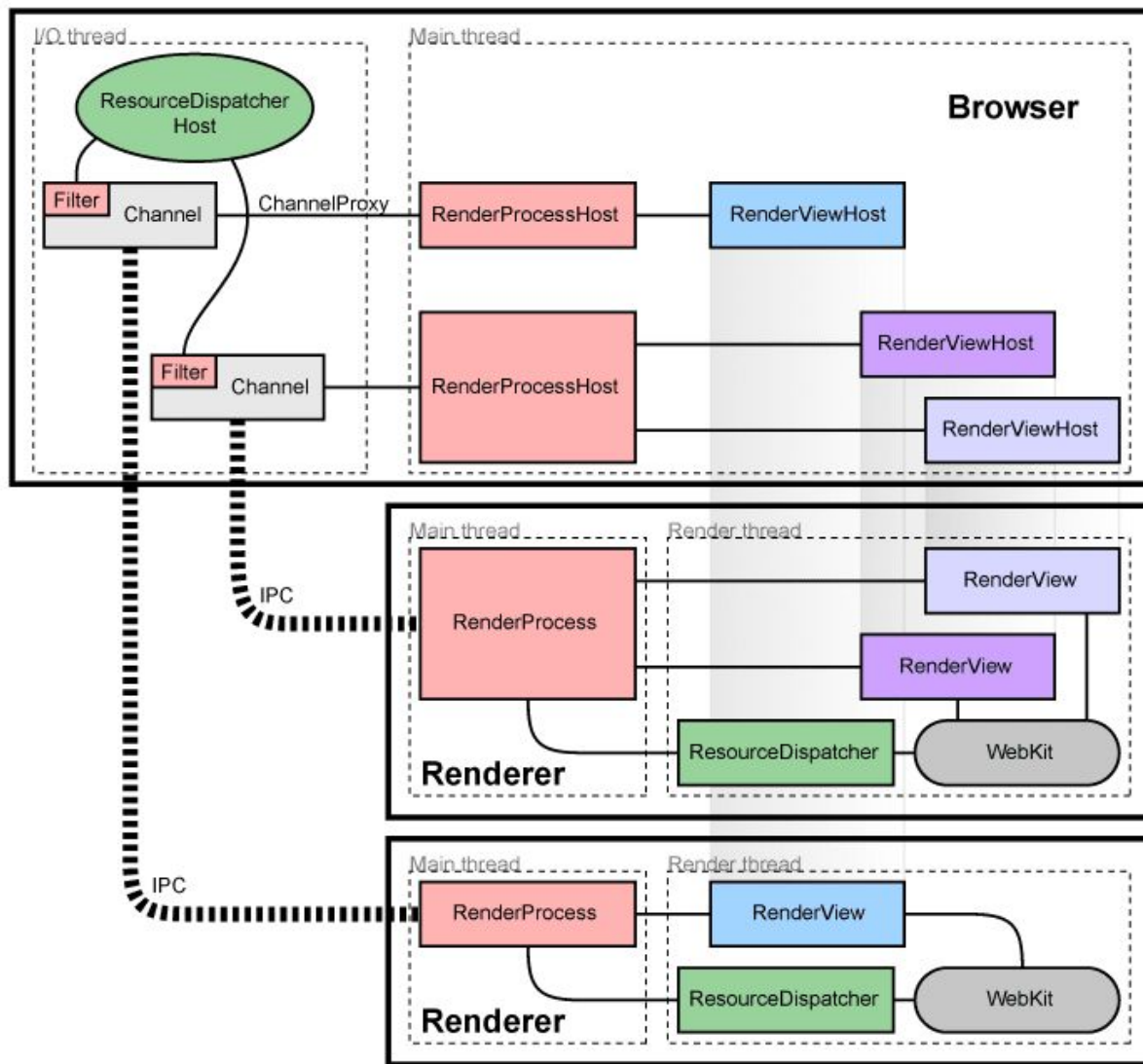
Multiprocess Architecture – Chrome Browser



- Many web browsers ran as single process (some still do)
 - If one website causes trouble, entire browser can hang or crash

- Google Chrome Browser is multiprocess with 3 different types of processes:
 - **Browser** process manages user interface, disk and network I/O
 - **Renderer** process renders web pages, deals with HTML, Javascript.
 - A new renderer created for each website opened
 - Runs in **sandbox** restricting disk and network I/O, minimizing effect of security exploits
 - **Plug-in** process for each type of plug-in

[Multi-process Architecture](#)



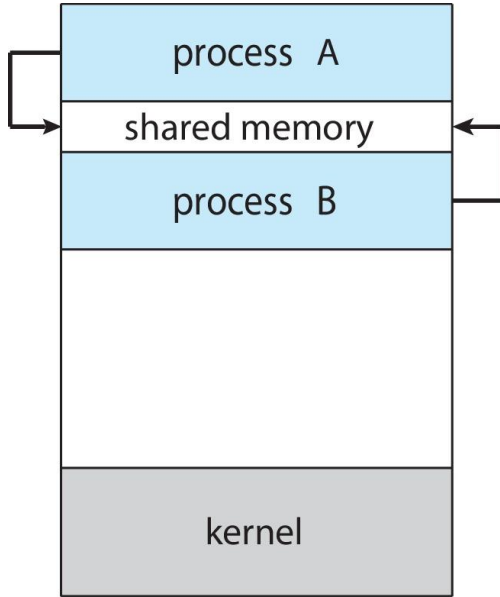
[Multi-process Architecture](#)

Need a mechanism to break isolation of processes...

How can we do IPC?

Two models of IPC

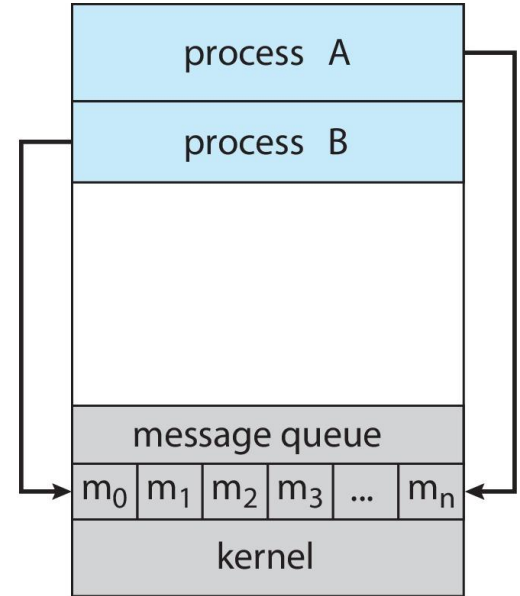
Shared memory.



Shared memory

- fast, efficient(no overhead), ideal for large data sharing
- needs synchronization, security, management

Message passing.



Message passing

- pros: secure, flexible(remote or local), error handling
- cons: Latency, overhead, complexity

What can go wrong?

A paradigm to study cooperating processes: producer-consumer problem

producer process produces information that is consumed by a *consumer* process

- Two variations:

- **unbounded-buffer** places no practical limit on the size of the buffer:
 - Producer never waits
 - Consumer waits if there is no buffer to consume

- **bounded-buffer** assumes that there is a fixed buffer size
 - Producer must wait if all buffers are full
 - Consumer waits if there is no buffer to consume

IPC – Shared Memory

- An area of memory shared among the processes that wish to communicate
- The communication is under the control of the users processes not the operating system.
- Major issues is to provide mechanism that will allow the user processes to synchronize their actions when they access shared memory.

Bounded-Buffer – Shared-Memory Solution

- Shared data

```
#define BUFFER_SIZE 10
typedef struct {
    . . .
} item;

item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

- Solution is correct, but can only use `BUFFER_SIZE-1` elements

Producer Process – Shared Memory

```
item next_produced;
```

```
while (true) {
```

```
    /* produce an item in next produced */
```

```
    while (((in + 1) % BUFFER_SIZE) == out)
```

```
        ; /* do nothing */
```

```
    buffer[in] = next_produced;
```

```
    in = (in + 1) % BUFFER_SIZE;
```

```
}
```

Consumer Process – Shared Memory

```
item next_consumed;
while (true) {
    while (in == out)
        ; /* do nothing */

    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;

    /* consume the item in next consumed */
}
```

- this example is a solution ONLY for 1-producer and 1-consumer (busy waiting)

This solution to the producer-consumer problem is from “Proving the Correctness of Multiprocess Programs,” by L. Lamport, IEEE Transactions on Software Engineering, SE-3(2) 1977: 125-143.

[Leslie Lamport](#)

What about Filling all the Buffers?

- consumer-producer problem fills **all** the buffers.
 - an integer **counter** that keeps track of the number of full buffers.
 - Initially, **counter** is set to 0.
 - The integer **counter** is incremented by the producer after it produces a new buffer.
 - The integer **counter** is and is decremented by the consumer after it consumes a buffer.

Producer

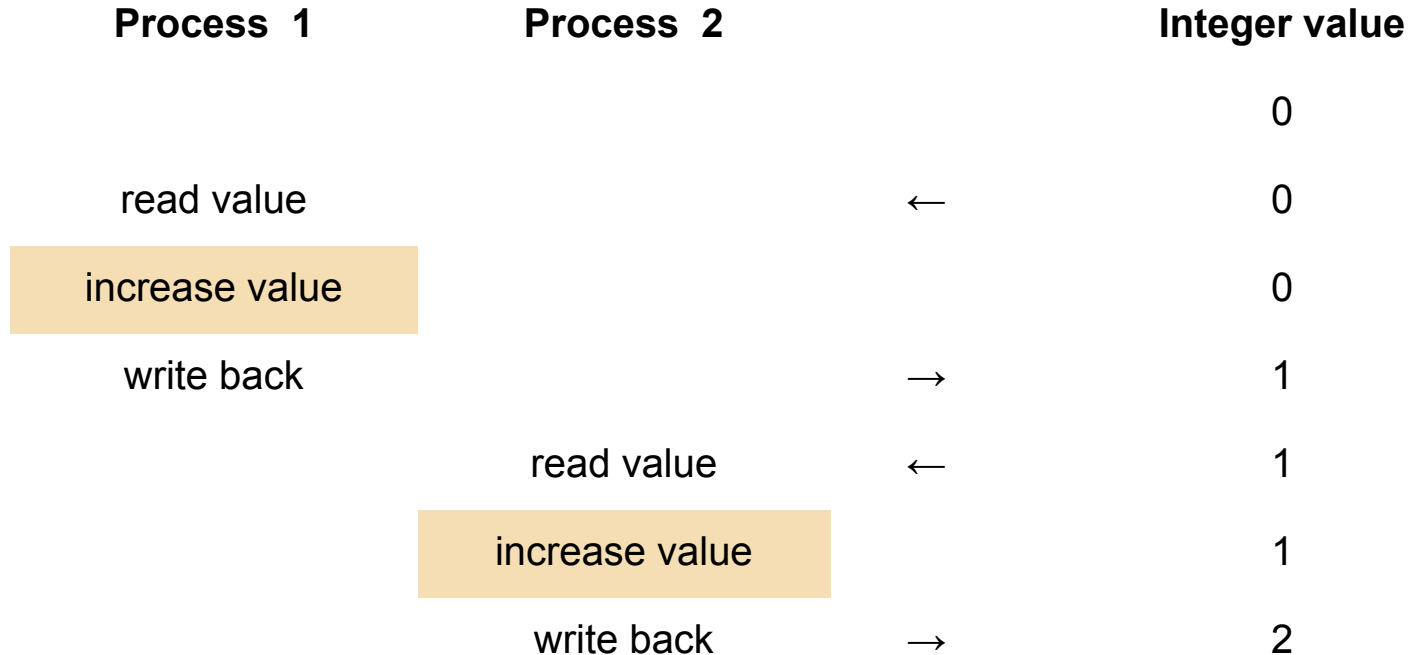
```
while (true) {  
    /* produce an item in next  
    produced */  
  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

Consumer

```
while (true) {  
    while (counter == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
  
    /* consume the item in next  
    consumed */  
}
```

Race Condition

[Race condition - Wikipedia](#) is the condition of an electronics, software, or other system where the system's substantive behavior is dependent on the sequence or timing of other uncontrollable events.



Race Condition

counter++ could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

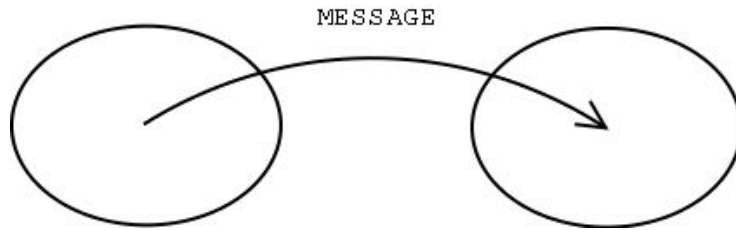
counter-- could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “count = 5” initially:

S0: producer execute	register1 = counter	{register1 = 5}
S1: producer execute	register1 = register1 + 1	{register1 = 6}
S2: consumer execute	register2 = counter	{register2 = 5}
S3: consumer execute	register2 = register2 - 1	{register2 = 4}
S4: producer execute	counter = register1	{counter = 6}
S5: consumer execute	counter = register2	{counter = 4}

IPC – Message Passing



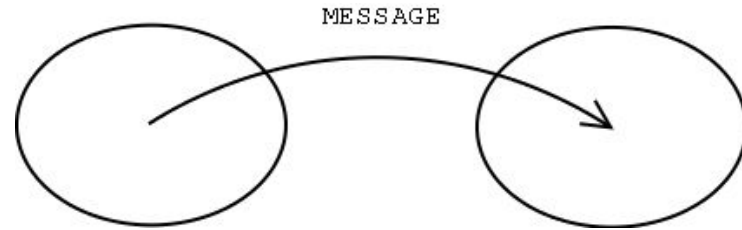
- Processes communicate with each other without resorting to shared variables
- IPC facility provides two operations:
 - `send(destination, &message);`
 - `receive(source, &message);`
- The *message* size is either fixed or variable

see tutorial:

<https://cs.lmu.edu/~ray/notes/messagepassing/>

Message Passing

- If processes P and Q wish to communicate, they need to:
 - Establish a ***communication link*** between them
 - Exchange messages via send/receive



Message Passing: Implementation issues

- **Many Implementation issues:**

- How are links established?
- Can a link be associated with more than two processes?
- How many links can there be between every pair of communicating processes?
- What is the capacity of a link?
- Is the size of a message that the link can accommodate fixed or variable?
- Is a link unidirectional or bi-directional?

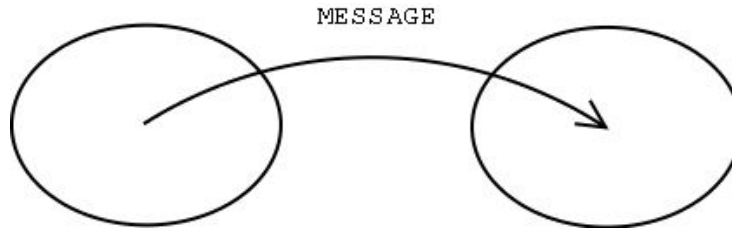
Implementation of Communication Link

- Physical:

- Shared memory
- Hardware bus
- Network

- Logical:

- Direct or indirect
- Synchronous or asynchronous
- Automatic or explicit buffering



Direct Communication

- Processes must name each other explicitly

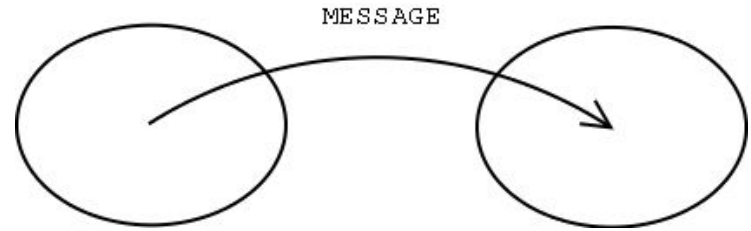
(**symmetric naming**):

- **send** (*P*, *message*)

- send a message to process P

- **receive**(*Q*, *message*)

- receive a message from process Q



Direct Communication

- Processes must name each other explicitly:
 - **send** (*P*, *message*) – send a message to process *P*
 - **receive**(*Q*, *message*) – receive a message from process *Q*

Asymmetric case:

send(*Q*, *message*)
receive(&*message*)

- Properties of communication link
 - Links are established automatically
 - A link is associated with exactly one pair of communicating processes
 - Between each pair there exists exactly one link
 - The link may be unidirectional, but is usually bi-directional

Indirect Communication

- Messages are directed and received from mailboxes (also referred to as ports)
 - Each mailbox has a unique id
 - Processes can communicate only if they share a mailbox
- Properties of communication link
 - Link established only if processes share a common mailbox
 - A link may be associated with many processes
 - Each pair of processes may share several communication links
 - Link may be unidirectional or bi-directional

Indirect Communication (Cont.)

- Operations

- Create a new mailbox (port)
- Send and receive messages through mailbox
- Delete a mailbox

- Primitives are defined as:

- **send**(*A, message*) – send a message to mailbox A
- **receive**(*A, message*) – receive a message from mailbox A

Indirect Communication (Cont.)

- Mailbox sharing

- P_1 , P_2 , and P_3 share mailbox A
- P_1 sends; P_2 and P_3 receive
- Who gets the message?

- Solutions

- Allow a link to be associated with at most two processes
- Allow only one process at a time to execute a receive operation
- Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Synchronization

Message passing may be either blocking or non-blocking

- **Blocking** is considered **synchronous**
 - **Blocking send** -- the sender is blocked until the message is received
 - **Blocking receive** -- the receiver is blocked until a message is available
- **Non-blocking** is considered **asynchronous**
 - **Non-blocking send** -- the sender sends the message and continue
 - **Non-blocking receive** -- the receiver receives:
 - A valid message, or
 - Null message
- Different combinations possible
 - If both send and receive are blocking, we have a **rendezvous**

Producer-Consumer: Message Passing

- Producer

```
message next_produced;

while (true) {

    /* produce an item in
    next_produced */

    send(next_produced) ;

}
```

- Consumer

```
message next_consumed;

while (true) {

    receive(next_consumed)

    /* consume the item in
    next_consumed */

}
```

Buffering messages

- Queue of messages attached to the link.
- Implemented in one of three ways
 1. **Zero capacity** – no messages are queued on a link.
Sender must wait for receiver (rendezvous)
 2. **Bounded capacity** – finite length of n messages
Sender must wait if link full
 3. **Unbounded capacity** – infinite length
Sender never waits

Examples of IPC Systems - POSIX

POSIX Shared Memory

Process first creates shared memory segment

```
fd = shm_open(name,  
              O_CREAT | O_RDWR,  
              0666);
```

- Also used to open an existing segment

Set the size of the object

```
ftruncate(fd, 4096);
```

Use `mmap()` to memory-map a file pointer to the shared memory object

Reading and writing to shared memory is done by using the pointer returned by `mmap()`.

IPC POSIX Producer

```
#include <stdio.h>
#include<stdlib.h>
#include<string.h>
#include<fcntl.h>
#include <sys/shm.h>
#include<sys/stat.h>
#include <sys/mman.h>

int main() {
    const int SIZE = 4096; /* the size (in bytes) of shared memory object */
    const char *name = "OS"; /* name of the shared memory object */

    /* strings written to shared memory */
    const char *message0 = "Hello", *message1 = "World!";
    int fd; /* shared memory file descriptor */

    char *ptr; /* pointer to shared memory object */

    /* create the shared memory object */
    fd = shm_open(name, O_CREAT | O_RDWR, 0666);
    /* configure the size of the shared memory object */
    ftruncate(fd, SIZE);
    /* memory map the shared memory object */
    ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);

    /* write to the shared memory object */
    sprintf(ptr, "%s", message0);
    ptr += strlen(message0);
    sprintf(ptr, "%s", message1);
    ptr += strlen(message1);
    return 0;
}
```

IPC POSIX Consumer

```
#include < sys/mman.h> int main() {
#include <fcntl.h>      /* the size (in bytes) of shared memory object */
#include <stdio.h>      const int SIZE = 4096;
#include <stdlib.h>     /* name of the shared memory object */
#include <sys/shm.h>    const char *name = "OS";
#include <sys/stat.h>   /* shared memory file descriptor */
                        int fd;
                        /* pointer to shared memory object */
                        char *ptr;
                        /* open the shared memory object */
                        fd = shm_open(name, O_RDONLY, 0666);
                        /* memory map the shared memory object */
                        ptr = (char *)mmap(0, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED,
                        fd, 0);
                        /* read from the shared memory object */
                        printf("%s", (char *)ptr);
                        /* remove the shared memory object */
                        shm_unlink(name);
                        return 0;
}
```

Examples of IPC Systems - Mach S

[Mach Project Publications and Related Documents](#)

[The GNU Mach Reference Manual](#)

see also **POSIX Message Queues** [3.6. Message Passing With Message Queues — Computer Systems Fundamentals](#)

- Mach communication is message based
 - Even system calls are messages
 - Each task gets two ports at creation - Kernel and Notify
 - Messages are sent and received using the `mach_msg()` function
- Ports needed for communication, created via
`mach_port_allocate()`
- Send and receive are flexible; for example four options if mailbox full:
 - Wait indefinitely
 - Wait at most n milliseconds
 - Return immediately
 - Temporarily cache a message

Mach Messages

```
#include <mach/mach.h>

struct message {
    mach_msg_header_t header;
    int data;
};

mach_port_t client;
mach_port_t server;
```


Mach Message Passing - Client

```
mach_msg_return_t mach_msg(  
    mach_msg_header_t *msg, mach_msg_option_t option,  
    mach_msg_size_t send_size, mach_msg_size_t rcv_size,  
    mach_port_t rcv_name, mach_msg_timeout_t timeout,  
    mach_port_t notify)  
  
/* Client Code */  
  
struct message message;  
  
// construct the header  
message.header.msgh_size = sizeof(message);  
message.header.msgh_remote_port = server;  
message.header.msgh_local_port = client;  
  
// send the message  
mach_msg(&message.header, // message header  
    MACH_SEND_MSG, // sending a message  
    sizeof(message), // size of message sent  
    0, // maximum size of received message - unnecessary  
    MACH_PORT_NULL, // name of receive port - unnecessary  
    MACH_MSG_TIMEOUT_NONE, // no time outs  
    MACH_PORT_NULL // no notify port  
);
```

Mach Message Passing - Server

```
/* Server Code */

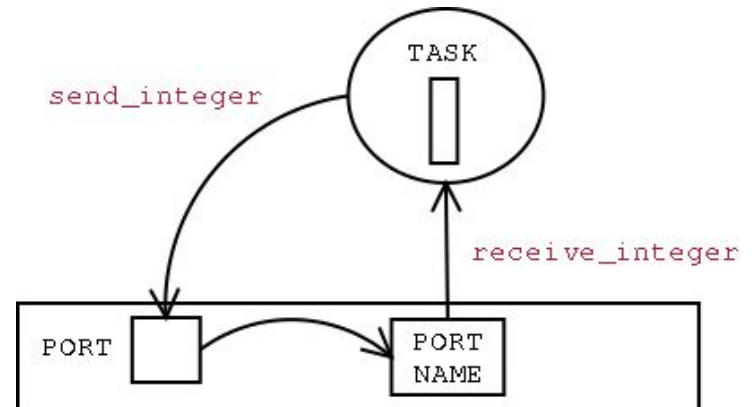
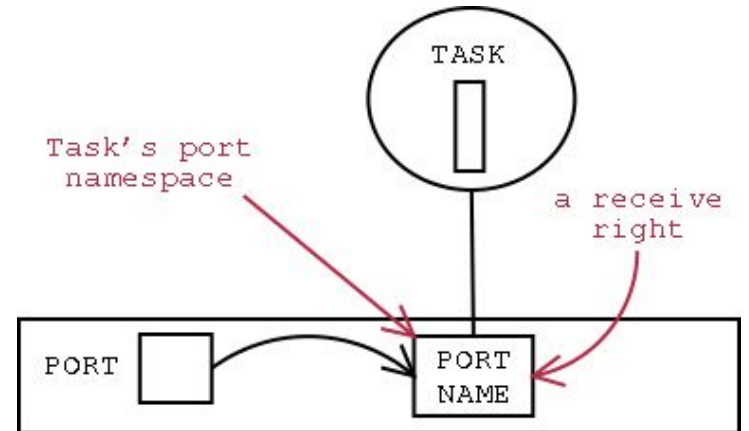
struct message message;

// receive the message
mach_msg(&message.header, // message header
        MACH_RCV_MSG, // sending a message
        0, // size of message sent
        sizeof(message), // maximum size of received message
        server, // name of receive port
        MACH_MSG_TIMEOUT_NONE, // no time outs
        MACH_PORT_NULL // no notify port
);
```

Mach another example

```
kern_return_t err;  
mach_port_t rcv_port;  
/*create a mach port*/  
err = mach_port_allocate(mach_task_self(),  
                        MACH_PORT_RIGHT_RECEIVE,  
                        &rcv_port);  
  
if (err != KERN_SUCCESS) {  
    perror("error : could not allocate any port\n");  
    exit(err);  
}  
  
struct integer_message {  
    mach_msg_header_t head;  
    mach_msg_type_t type;  
    int inline_integer;  
};
```

[Mach's IPC Basic Concepts](#)



```

void send_integer( mach_port_t destination, int i ){
    kern_return_t err;

    struct integer_message message;
    /* (i.a) Fill the header fields : */
    message.head.msgh_bits =
MACH_MSGH_BITS_REMOTE(MACH_MSG_TYPE_MAKE_SEND);
    message.head.msgh_size = sizeof( struct integer_message );
    message.head.msgh_local_port = MACH_PORT_NULL;
    message.head.msgh_remote_port = destination;

    struct integer_message{
        mach_msg_header_t head;
        mach_msg_type_t type;
        int inline_integer;
    };

    /* (i.b) Explain the message type ( an integer ) */
    message.type.msgt_name = MACH_MSG_TYPE_INTEGER_32;
    message.type.msgt_size = 32;
    message.type.msgt_number = 1;
    message.type.msgt_inline = TRUE;
    message.type.msgt_longform = FALSE;
    message.type.msgt_deallocate = FALSE;
    /* message.type.msgt_unused = 0; */ /* not needed, I think */
    /* (i.c) Fill the message with the given integer : */
    message.inline_integer = i;
    /* (ii) Send the message : */

```

Mach's IPC Basic Concepts

```

void send_integer( mach_port_t destination, int i ){
    ...
    /* (ii) Send the message : */
    err = mach_msg( &(message.head), MACH_SEND_MSG,
                    message.head.msgh_size, 0, MACH_PORT_NULL,
                    MACH_MSG_TIMEOUT_NONE, MACH_PORT_NULL );
    /* (iii) Analysis of the error code;
    if succes, print and acknowledge message and return */
    if( err == MACH_MSG_SUCCESS ){
        printf( "success: the message was queued\n" );
    }
    else{
        perror( "error: some unexpected error occurred!\n");
        exit(err);
    }

    return;
}

```

Receive integer

```
void receive_integer(mach_port_t source, int *ip) {
    kern_return_t err;

    struct integer_message message;

    /* (i) Fill the little thing we know about the message : */
    /* message.head.msgh_size = sizeof(struct integer_message ); */

    /* (ii) Receive the message : */
    err = mach_msg(&(message.head), MACH_RCV_MSG, 0, message.head.msgh_size,
                  source, MACH_MSG_TIMEOUT_NONE, MACH_PORT_NULL);

    if (err == MACH_MSG_SUCCESS) {
        printf("success: the message was received\n");
    } else {
        perror("error: Some unexpected error occurred\n");
        exit(err);
    }

    *ip = message.inline_integer;

    return;
}
```

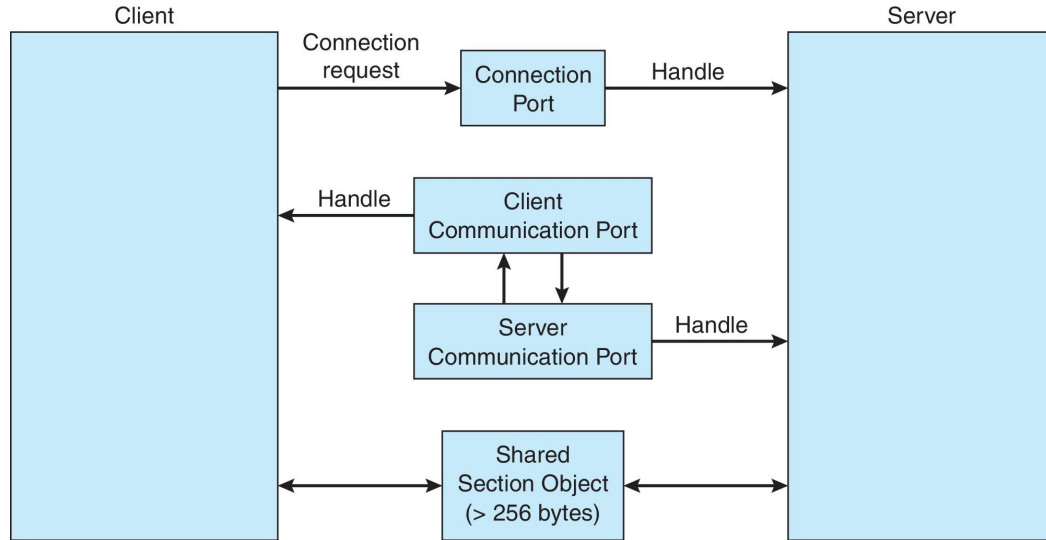
[Mach's IPC Basic Concepts](#)

see also [Mach ports - Darling Docs](#) and references therein

Examples of IPC Systems – Windows

- Message-passing centric via **advanced local procedure call (LPC)** facility
 - Only works between processes on the same system
 - Uses ports (like mailboxes) to establish and maintain communication channels

Local Procedure Calls in Windows



Communication works as follows:

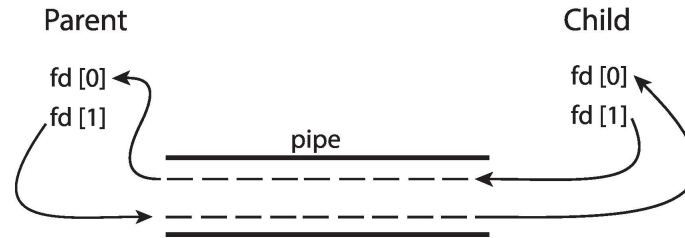
- Server(subsystem) processes publish **connection port** objects that are visible to all processes.
- The client requests a connection to named port.
- The server creates two private **communication ports**
 - **client communication port**
 - returns the handle to the client.
 - **server communication port**
- The client and server use the corresponding port handle to send messages or callbacks and to listen for replies.

Pipes

- allows two or more processes to communicate with each other by creating a unidirectional or bidirectional channel between them
- Issues:
 - Is communication unidirectional or bidirectional?
 - In the case of two-way communication, is it half or full-duplex?
 - Must there exist a relationship (i.e., **parent-child**) between the communicating processes?
 - Can the pipes be used over a network?
- **Ordinary pipes** – cannot be accessed from outside the process that created it. Typically, a parent process creates a pipe and uses it to communicate with a child process that it created.
- **Named pipes** – can be accessed without a parent-child relationship.

Ordinary Pipes

- Ordinary Pipes allow communication in standard producer-consumer style
- Producer writes to one end (the **write-end** of the pipe)
- Consumer reads from the other end (the **read-end** of the pipe)
- Ordinary pipes are therefore unidirectional
- Require parent-child relationship between communicating processes



- Windows calls these **anonymous pipes**

Named Pipes

- Named Pipes are more powerful than ordinary pipes
- Communication is bidirectional
- No parent-child relationship is necessary between the communicating processes
- Several processes can use the named pipe for communication
- Provided on both UNIX and Windows systems

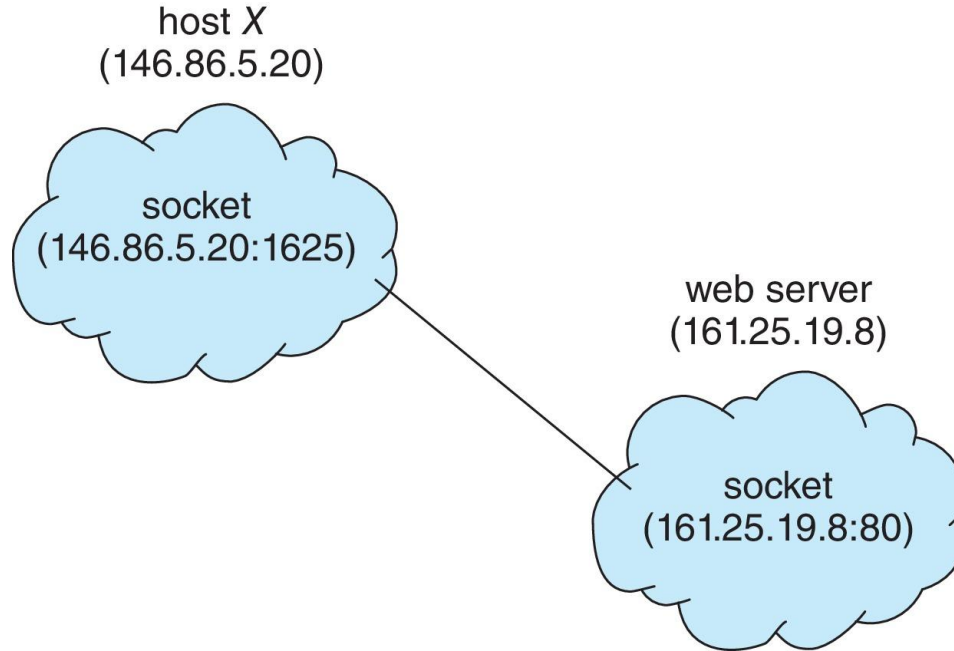
Communications in Client-Server Systems

- Sockets
- Remote Procedure Calls

Sockets

- A **socket** is defined as an endpoint for communication
- Concatenation of IP address and **port** – a number included at start of message packet to differentiate network services on a host
- The socket **161.25.19.8:1625** refers to port **1625** on host **161.25.19.8**
- Communication consists between a pair of sockets
- All ports below 1024 are ***well known***, used for standard services
- Special IP address 127.0.0.1 (**loopback**) to refer to system on which process is running

Socket Communication



Socket Programming in Java

- Three types of sockets
 - **Connection-oriented (TCP)**
 - **Connectionless (UDP)**
 - **MulticastSocket** class—
data can be sent to multiple recipients
- Consider this “Date” server in Java:

see also [Berkeley sockets - Wikipedia](#) or
System programming lecture notes for C
examples

```
import java.net.*;
import java.io.*;

public class DateServer
{
    public static void main(String[] args) {
        try {
            ServerSocket sock = new ServerSocket(6013);

            /* now listen for connections */
            while (true) {
                Socket client = sock.accept();

                PrintWriter pout = new
                    PrintWriter(client.getOutputStream(), true);

                /* write the Date to the socket */
                pout.println(new java.util.Date().toString());

                /* close the socket and resume */
                /* listening for connections */
                client.close();
            }
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```

The implementation of Date client

```
import java.net.*;
import java.io.*;

public class DateClient
{
    public static void main(String[] args) {
        try {
            /* make connection to server socket */
            Socket sock = new Socket("127.0.0.1",6013);

            InputStream in = sock.getInputStream();
            BufferedReader bin = new
                BufferedReader(new InputStreamReader(in));

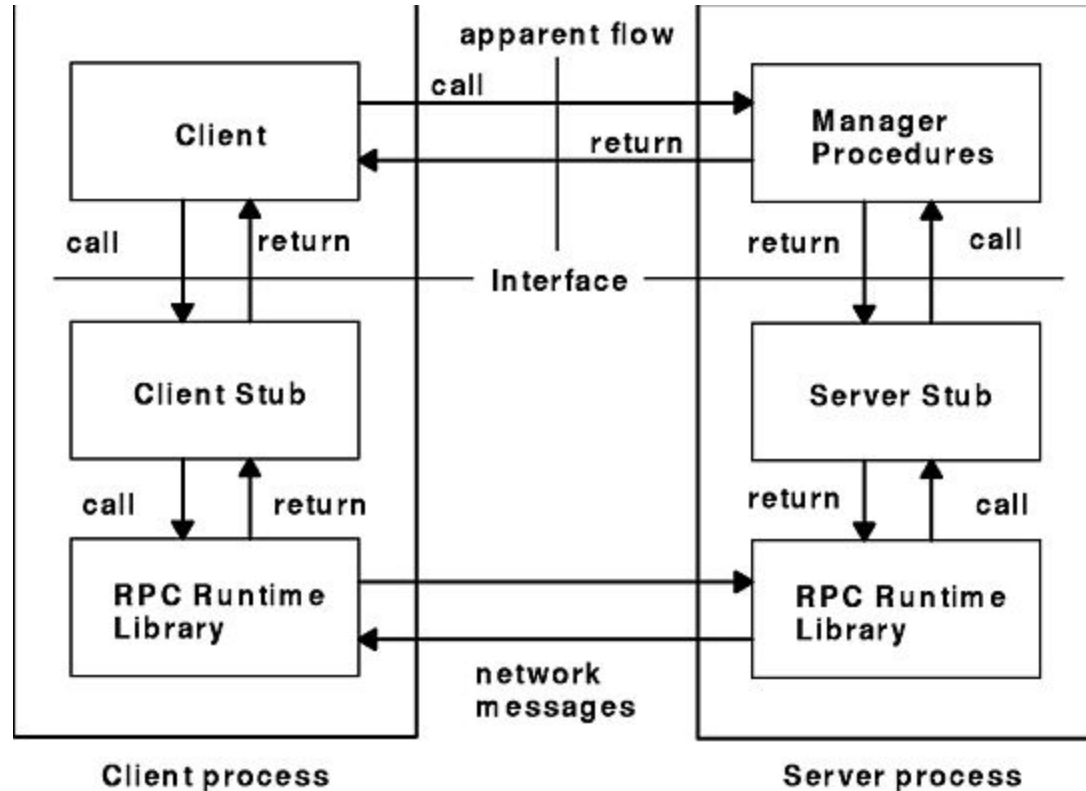
            /* read the date from the socket */
            String line;
            while ( (line = bin.readLine()) != null)
                System.out.println(line);

            /* close the socket connection*/
            sock.close();
        }
        catch (IOException ioe) {
            System.err.println(ioe);
        }
    }
}
```


Remote Procedure Calls

- Remote procedure call (RPC) abstracts procedure calls between processes on networked systems
 - high level protocol that programs can use to request services from other programs
 - request-response based protocol
 - Again uses ports for service differentiation
- **Stubs** – client-side proxy for the actual procedure on the server
 - On Windows, stub code compile from specification written in **Microsoft Interface Definition Language (MIDL)**

An example model of RPC flow



Remote Procedure Call Flow

Sequence of events

- The client calls the client stub.
 - The call is a local procedure call,
 - with parameters pushed on to the stack in the normal way.
- The client stub packs the parameters into a message and makes a system call to send the message.
 - Packing the parameters is called **marshalling**.
- The client's local operating system sends the message from the client machine to the server machine.
- The local OS on the server machine passes the incoming packets to the server stub.
- The server stub unpacks the parameters from the message.
 - Unpacking the parameters is called **unmarshalling**.
- Finally, the server stub calls the server procedure.

The reply traces the same steps in the reverse direction.

Remote Procedure Calls (Cont.)

- Data representation handled via **External Data Representation (XDR)** format to account for different architectures
 - **Big-endian** and **little-endian**
- Remote communication has more failure scenarios than local
 - Messages can be delivered ***exactly once*** rather than ***at most once***
- OS typically provides a rendezvous (or **matchmaker**) service to connect client and server

Summary

we learned...

process creation

IPC

Different IPC examples and mechanisms

- shared memory
 - problems
- message passing
 - direct communication
 - indirect communication

- Is communication synchronous or asynchronous?
- Is naming symmetric or asymmetric?
- Do we name the parties or do we name the channels?
- Is the messaging client-server-ish or a free-for-all bulletin board?

[Message Passing](#)